

Bases de données et langage SQL

SAÉ S104

Prénom : Achour
Nom : HAMOU YAHIA
Groupe : Mercure
Promotion : BUT INFO
Année : 2025/2026

2.1 Script manuel de création de la base de données

1) Script SQL de créations des tables

```
CREATE TABLE regions (  
    code VARCHAR PRIMARY KEY,  
    nom VARCHAR NOT NULL ) ;  
  
CREATE TABLE departements (  
    code VARCHAR PRIMARY KEY,  
    region VARCHAR NOT NULL REFERENCES regions (code),  
    nom VARCHAR NOT NULL ) ;  
  
CREATE TABLE communes (  
    code VARCHAR PRIMARY KEY,  
    departement VARCHAR NOT NULL REFERENCES departements (code),  
    nom VARCHAR NOT NULL ) ;  
  
CREATE TABLE sites (  
    idSite VARCHAR PRIMARY KEY,  
    nom VARCHAR NOT NULL,  
    codeCommune VARCHAR NOT NULL REFERENCES communes (code),  
    dateDeclaration DATE NOT NULL,  
    typeEau VARCHAR NOT NULL,  
    longitude NUMERIC NOT NULL,  
    latitude NUMERIC NOT NULL ) ;  
  
CREATE TABLE evenements (  
    idEvenement INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    idSite VARCHAR NOT NULL REFERENCES sites(idSite),  
    evenement VARCHAR NOT NULL,  
    debut DATE NOT NULL,  
    fin DATE,  
    mesure VARCHAR ) ;  
  
CREATE TABLE analyses (  
    idAnalyse INTEGER GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    idSite VARCHAR NOT NULL REFERENCES sites(idSite),  
    datePrelevement DATE NOT NULL,  
    enterocoques INTEGER,  
    escherichia INTEGER ) ;
```

2.2 Modélisation et script de création « avec AGL »

1. Illustrations comparatives cours/AGL commentée d'une association fonctionnelle.

Image du cours :

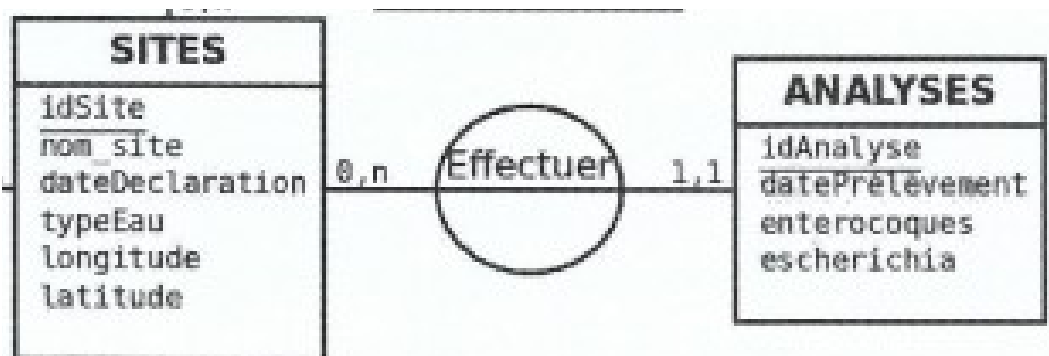
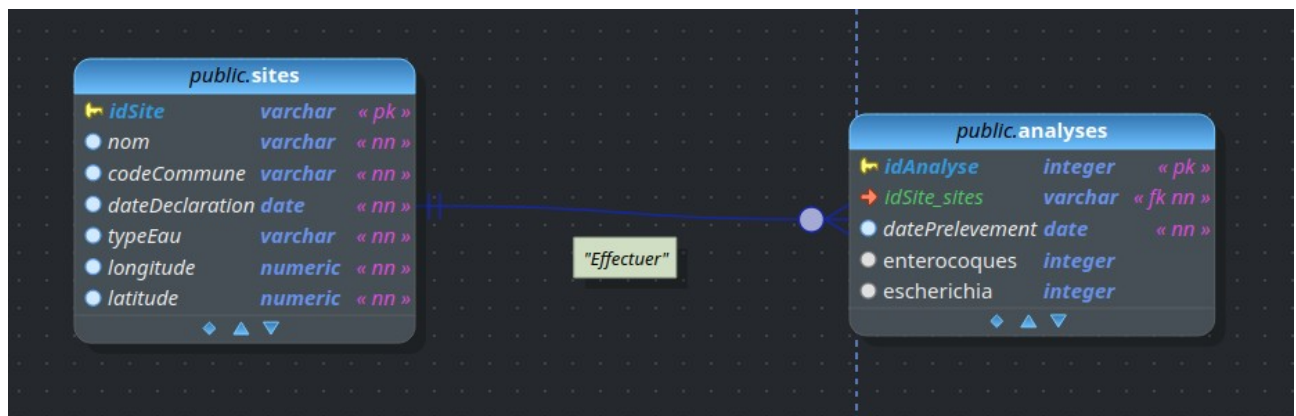


Image du logiciel (PgModeler) :



Commentaire :

Dans le modèle du cours, on remarque que l'association est représenté par une bulle qui relie les deux entités, avec comme cardinalité (0,n) et (1,1). Ce qui signifie qu'une analyse ne peut avoir qu'un seul site.

Alors que dans le modèle du logiciel, cette bulle disparaît pour laisser place à un trait qui crée un lien entre les deux entités pour ajouter une colonne clé étrangère dans la table analyses.

2. Illustrations comparatives cours/AGL commentée d'une association mailler.

Image du cours :

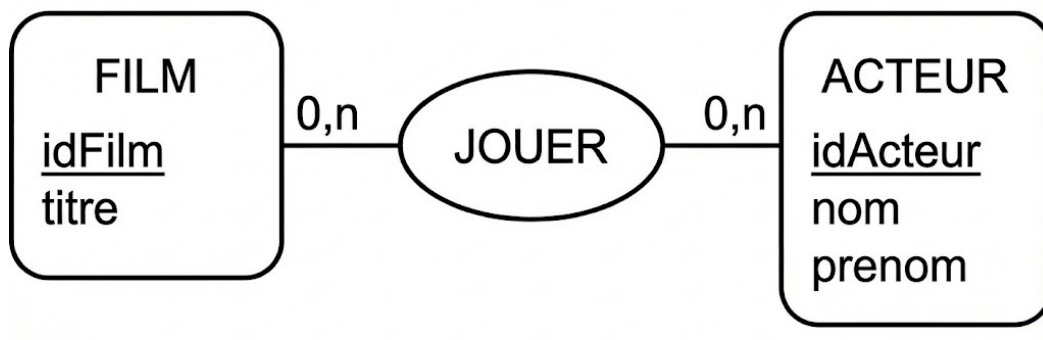


Image du logiciel (PgModeler) :

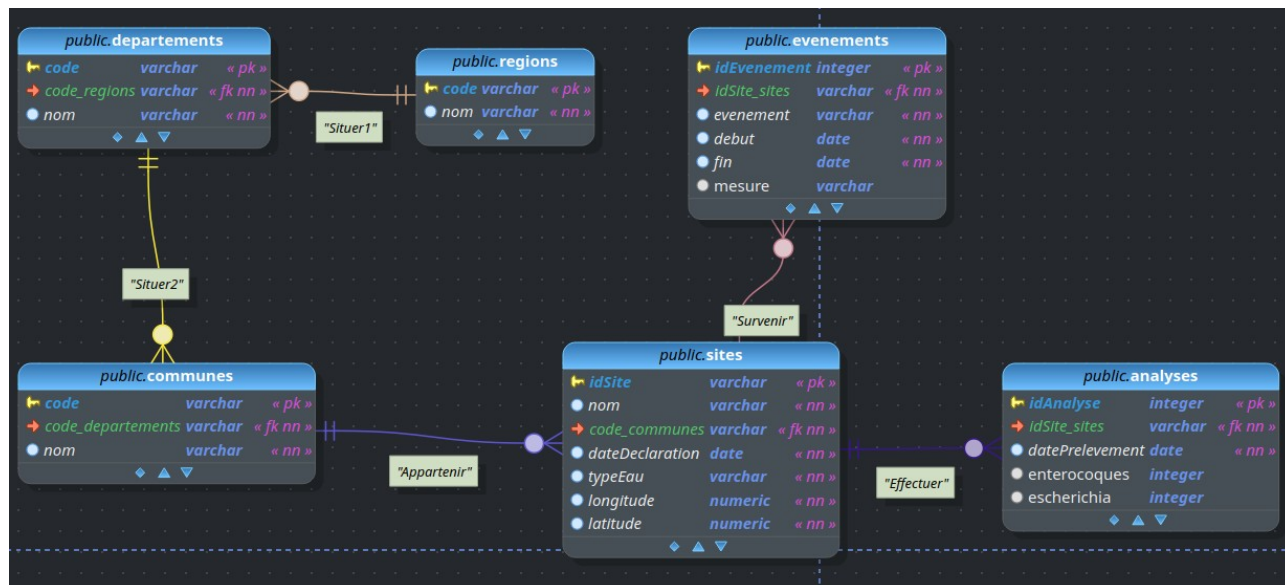


Commentaire :

Contrairement à l'illustration précédente, le modèle du cours d'une association fonctionnelle ne change pas vraiment par rapport à celle d'une association mailler, hormis, bien sur, les cardinalité (0,n). Là où, le modèle du logiciel PgModeler change complètement en laissant place à une nouvelle table à la place de la bulle qui relie ces deux entités.

On note aussi que contrairement au modèle précédent, dans celui ci on a la possibilité de convertir notre nouvelle table Jouer en un trait de la même manière que le précédent.

3) Modèle physique de données réalisé avec l'AGL.



4) Script SQL de création de tables généré automatiquement par l'AGL.

```
-- Database generated with pgModeler (PostgreSQL Database Modeler).
-- pgModeler version: 1.1.6
-- PostgreSQL version: 17.0
-- Project Site: pgmodeler.io
-- Model Author: ---

-- Database creation must be performed outside a multi lined SQL file.
-- These commands were put in this file only as a convenience.
--
-- object: new_database | type: DATABASE --
-- DROP DATABASE IF EXISTS new_database;
CREATE DATABASE new_database;
-- ddl-end --

-- object: public.regions | type: TABLE --
-- DROP TABLE IF EXISTS public.regions CASCADE;
CREATE TABLE public.regions (
    code varchar NOT NULL,
    nom varchar NOT NULL,
    CONSTRAINT regions_pk PRIMARY KEY (code)
);
-- ddl-end --
ALTER TABLE public.regions OWNER TO postgres;
-- ddl-end --

-- object: public.departements | type: TABLE --
-- DROP TABLE IF EXISTS public.departements CASCADE;
CREATE TABLE public.departements (
    code varchar NOT NULL,
    code_regions varchar NOT NULL,
    nom varchar NOT NULL,
    CONSTRAINT departements_pk PRIMARY KEY (code)
);
```

```

-- ddl-end --
ALTER TABLE public.departements OWNER TO postgres;
-- ddl-end --

-- object: regions_fk | type: CONSTRAINT --
-- ALTER TABLE public.departements DROP CONSTRAINT IF EXISTS regions_fk
CASCADE;
ALTER TABLE public.departements ADD CONSTRAINT regions_fk FOREIGN KEY
(code_regions)
REFERENCES public.regions (code) MATCH FULL
ON DELETE RESTRICT ON UPDATE CASCADE;
-- ddl-end --

-- object: public.communes | type: TABLE --
-- DROP TABLE IF EXISTS public.communes CASCADE;
CREATE TABLE public.communes (
    code varchar NOT NULL,
    code_departements varchar NOT NULL,
    nom varchar NOT NULL,
    CONSTRAINT communes_pk PRIMARY KEY (code)
);
-- ddl-end --
ALTER TABLE public.communes OWNER TO postgres;
-- ddl-end --

-- object: departements_fk | type: CONSTRAINT --
-- ALTER TABLE public.communes DROP CONSTRAINT IF EXISTS departements_fk
CASCADE;
ALTER TABLE public.communes ADD CONSTRAINT departements_fk FOREIGN KEY
(code_departements)
REFERENCES public.departements (code) MATCH FULL
ON DELETE RESTRICT ON UPDATE CASCADE;
-- ddl-end --

-- object: public.sites | type: TABLE --
-- DROP TABLE IF EXISTS public.sites CASCADE;
CREATE TABLE public.sites (
    "idSite" varchar NOT NULL,
    nom varchar NOT NULL,
    code_communes varchar NOT NULL,
    "dateDeclaration" date NOT NULL,
    "typeEau" varchar NOT NULL,
    longitude numeric NOT NULL,
    latitude numeric NOT NULL,
    CONSTRAINT sites_pk PRIMARY KEY ("idSite")
);
-- ddl-end --
ALTER TABLE public.sites OWNER TO postgres;
-- ddl-end --

-- object: public.analyses | type: TABLE --
-- DROP TABLE IF EXISTS public.analyses CASCADE;
CREATE TABLE public.analyses (
    "idAnalyse" integer NOT NULL GENERATED ALWAYS AS IDENTITY ,
    "idSite_sites" varchar NOT NULL,
    "datePrelevement" date NOT NULL,
    enterocoques integer,
    escherichia integer,

```

```

        CONSTRAINT analyses_pk PRIMARY KEY ("idAnalyse")
    );
    -- ddl-end --
    ALTER TABLE public.analyses OWNER TO postgres;
    -- ddl-end --

    -- object: communes_fk | type: CONSTRAINT --
    -- ALTER TABLE public.sites DROP CONSTRAINT IF EXISTS communes_fk
    CASCADE;
    ALTER TABLE public.sites ADD CONSTRAINT communes_fk FOREIGN KEY
    (code_communes)
    REFERENCES public.communes (code) MATCH FULL
    ON DELETE RESTRICT ON UPDATE CASCADE;
    -- ddl-end --

    -- object: sites_fk | type: CONSTRAINT --
    -- ALTER TABLE public.analyses DROP CONSTRAINT IF EXISTS sites_fk
    CASCADE;
    ALTER TABLE public.analyses ADD CONSTRAINT sites_fk FOREIGN KEY
    ("idSite_sites")
    REFERENCES public.sites ("idSite") MATCH FULL
    ON DELETE RESTRICT ON UPDATE CASCADE;
    -- ddl-end --

    -- object: public.evenements | type: TABLE --
    -- DROP TABLE IF EXISTS public.evenements CASCADE;
    CREATE TABLE public.evenements (
        "idEvenement" integer NOT NULL GENERATED ALWAYS AS IDENTITY ,
        "idSite_sites" varchar NOT NULL,
        evenement varchar NOT NULL,
        debut date NOT NULL,
        fin date NOT NULL,
        mesure varchar,
        CONSTRAINT evenements_pk PRIMARY KEY ("idEvenement")
    );
    -- ddl-end --
    ALTER TABLE public.evenements OWNER TO postgres;
    -- ddl-end --

    -- object: sites_fk | type: CONSTRAINT --
    -- ALTER TABLE public.evenements DROP CONSTRAINT IF EXISTS sites_fk
    CASCADE;
    ALTER TABLE public.evenements ADD CONSTRAINT sites_fk FOREIGN KEY
    ("idSite_sites")
    REFERENCES public.sites ("idSite") MATCH FULL
    ON DELETE RESTRICT ON UPDATE CASCADE;
    -- ddl-end --

```

5) Discussion sur les différences entre les scripts produits manuellement et automatiquement.

On remarque facilement une grande différence entre mon script réalisé manuellement dans la première partie et celui généré par l'AGL pgModeler qui met en évidence plusieurs différences.

Tout d'abord, dans le script manuel, le nom des colonnes demandés dans le modèle conceptuel seront toujours respecté, comme `codeCommune` ou `idSite`. L'AGL, lui, génère automatiquement des noms composites pour les clés étrangères afin de toute les différencier, ce qui donne des résultats comme `idSite_sites` ou `code_communes`. Même si elle reste valide, cette modification s'éloigne du plan initiale du sujet.

Ensuite, dans le script manuel, les clés étrangères sont déclarées à l'intérieur de la commande de création de la table. Cette méthode est plus rapide mais impose une contrainte sur l'ordre d'exécution, car les tables parentes doivent d'abord être créées avant les tables enfants. Alors que, le script de l'AGL agit différemment : d'abord il va créer toutes les tables, puis il va utiliser l'instruction `ALTER TABLE` pour ajouter des clés étrangères à la fin.

L'AGL sécurise aussi les noms en ajoutant des guillemets doubles autour des identifiants comportant des majuscules, comme `"idSite"` ou `"dateDeclaration"`, pgModeler fait en sorte de respecter les mots exactes. Le script manuel, écrit en SQL standard sans guillemets, laisse le PostgreSQL convertir tous les identifiants en minuscules. Avec l'AGL, elle obligera l'utilisateur à utiliser des guillemets lors de l'interrogation de la base, ce qui peut rendre plus fatigant l'écriture des requêtes.

Et enfin, le script automatique est bien mieux construit et sécurisé. Il écrit des commandes de nettoyage `DROP TABLE IF EXISTS` qui permet de supprimer les tables déjà existent si l'on relance le script une deuxième fois, là où le script manuel se repose uniquement sur les comportements de base du script de base de données.

2.3 Peuplement des tables

1) Script de peuplement de la base de données.

```
CREATE TEMPORARY TABLE Script_BD (  
    code_departement text,  
    nom_departement text,  
    code_region text,  
    nom_region text  
);  
\copy Script_BD FROM '/tmp/departements-france.csv' WITH (FORMAT csv, HEADER  
true, DELIMITER ',');  
  
INSERT INTO public.regions (code, nom)  
SELECT DISTINCT code_region, nom_region  
FROM Script_BD  
WHERE code_region IS NOT NULL;  
  
INSERT INTO public.departements (code, nom, code_regions)  
SELECT code_departement, nom_departement, code_region  
FROM Script_BD;  
  
DROP TABLE Script_BD;  
  
CREATE TEMPORARY TABLE temp_sites_import (  
    saison text,  
    region text,  
    dept_code_3chiffres text,  
    id_site text,  
    id_site_prec text,  
    evolution text,  
    nom_site text,  
    code_insee_commune text,  
    nom_commune text,  
    date_declaration text,  
    type_eau text,  
    longitude text,  
    latitude text  
);  
  
\copy temp_sites_import FROM '/tmp/liste-des-sites-de-baignade-saison-  
balneaire-2024.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING  
'LATIN1');  
  
INSERT INTO public.communes (code, nom, code_departements)  
SELECT DISTINCT  
    code_insee_commune,  
    nom_commune,  
    CASE  
        WHEN length(dept_code_3chiffres) = 3 AND left(dept_code_3chiffres, 1) =  
'0'  
        THEN substring(dept_code_3chiffres, 2, 2)  
        ELSE dept_code_3chiffres  
    END  
FROM temp_sites_import  
WHERE code_insee_commune IS NOT NULL  
ON CONFLICT (code) DO NOTHING;  
  
INSERT INTO public.sites ("idSite", nom, code_communes, "dateDeclaration",  
"typeEau", longitude, latitude)  
SELECT
```

```

        id_site,
        nom_site,
        code_insee_commune,
        CASE
            WHEN date_declaration IS NULL OR date_declaration = '' THEN '2024-01-
01'::date
            ELSE to_date(date_declaration, 'DD/MM/YYYY')
        END,
        type_eau,
        REPLACE(longitude, ',', '. '::numeric,
        REPLACE(latitude, ',', '. '::numeric
FROM temp_sites_import
WHERE id_site IS NOT NULL;

DROP TABLE temp_sites_import;

CREATE TEMPORARY TABLE temp_analyses (
    saison text,
    region text,
    dept text,
    code_site text,
    date_prel text,
    res_entero text,
    res_ecoli text,
    statut text,
    vide1 text,
    vide2 text,
    vide3 text
);

\copy temp_analyses FROM '/tmp/saison-balneaire-2024-resultats-danalyses.csv'
WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'LATIN1');

INSERT INTO public.analyses ("idSite_sites", "datePrelevement", enterocoques,
escherichia)
SELECT
    code_site,
    TO_DATE(date_prel, 'DD/MM/YYYY'),
    NULLIF(REGEXP_REPLACE(res_entero, '^[0-9]', '', 'g'), '')::integer,
    NULLIF(REGEXP_REPLACE(res_ecoli, '^[0-9]', '', 'g'), '')::integer
FROM temp_analyses
WHERE code_site IS NOT NULL
    AND date_prel IS NOT NULL;

DROP TABLE temp_analyses;

CREATE TEMPORARY TABLE temp_evenements (
    saison text,
    region text,
    dept text,
    code_site text,
    type_event text,
    date_debut text,
    date_fin text,
    mesures text
);

\copy temp_evenements FROM '/tmp/saison-balneaire-2024-informations-sur-la-
saison.csv' WITH (FORMAT csv, HEADER true, DELIMITER ';', ENCODING 'LATIN1');

INSERT INTO public.evenements ("idSite_sites", evenement, debut, fin, mesure)
SELECT
    code_site,
    type_event,

```

```

        TO_DATE(date_debut, 'DD/MM/YYYY'),
        TO_DATE(date_fin, 'DD/MM/YYYY'),
        mesures
FROM temp_evenements
WHERE code_site IS NOT NULL;

DROP TABLE temp_evenements;

```

2) Description commentée des différentes étapes de votre script de peuplement.

Pour réaliser le peuplement de la base de données, le mieux a été de créer des tables temporaires. Cela permet d'éviter les erreurs lors de la première importation, car les fichiers csv contiennent des données souvent mal formatées que la base de donnée n'aurait pas réussi à accepter. Alors j'ai chargé ces fichiers dans des tables temporaires pour ensuite les modifier correctement à l'aide de commande SQL, de sorte à pouvoir les entrer dans les tables finales.

Concernant le fichier CSV des départements, elle contient aussi les noms des régions. Le problème est que le nom d'une région est répété plusieurs fois sur chaque ligne de ses départements (par exemple, "Bretagne" apparaît 4 fois). Pour remplir la table regions, j'ai donc dû séparer ces informations avec la commande distinct, qui permet d'éliminer les doublons et ne garde qu'un seul exemplaire de chaque région. Une fois que la liste des régions a été proprement créée, j'ai pu importer les départements en faisant le lien avec le code de leur région.

Ensuite, le fichier des sites de baignade n'était pas compatible avec la base de données. J'ai donc dû effectuer quelques corrections au moment de l'importation : forcer la lecture en 'Latin1' du fichier, sinon les accents (é, à, ô) se transformaient en des symboles illisibles ; remplacer les virgules par des points car, en informatique, les nombres décimaux s'écrivent avec un point ; inverser les dates car dans le fichier CSV elles étaient écrites (jour/mois/année), sauf que les bases de données les stock à l'envers ; et certains codes étaient écrit avec 3 chiffres, j'ai donc du les modifier pour qu'elle corresponde bien à la table de département.

Enfin, il y avait deux derniers problèmes avec les fichiers de résultats d'analyses. Le premier était les quantités de bactéries : elles étaient écrites avec des symboles mathématiques (comme <15 ou >100). La base de données

ne veut que des chiffres. J'ai donc utilisé une commande pour supprimer automatiquement les symboles et ne garder que des nombres. Le deuxième problème était que certaines lignes du fichier CSV étaient mal finies et contenaient des points-virgules en trop à la fin, ce qui faisait croire à l'ordinateur qu'il y avait des colonnes en plus. Pour éviter que l'importation ne plante, j'ai ajouté des colonnes vides dans ma table temporaire pour absorber les colonnes en trop.